Alicia McMurchie
3/9/2026
Foundations Of Databases & SQL Programming
Assignment 07

Functions in SQL Server

Introduction

Functions are a specific type of saved portion of SQL code that can be called in relational databases like SQL Server, and have the advantage of allowing a data engineer to utilize parameters. While SQL Server includes a number of pre-existing functions, users can also create user-defined functions. These are useful for creating unique and potentially complex tables for reporting, as well as performing calculations. There are several types of functions a data engineer can create and use, including scalar, inline, and multi-statement. Data engineers may need to use all three, often in coordination, to create the reports and calculations required by their organization.

Use Cases of User-Defined Functions

While SQL-server includes many built-in functions, data engineers may need to create their own custom user-defined functions. This can happen when the built-in functions in SQL Server are not sufficient for tasks such as completing complex calculations to populate new columns in a table or view, reporting, Extract-Transform-Load processes, and creating check constraints.

When used for reporting, user-defined functions can be used to return a table of values. This can be useful when a user would like to see only a set of results. To illustrate, the example below shows a user-defined function to return a table based on a Key Performance Indicator (KPI) parameter than can be set by the user (Figure 1).

```
CREATE FUNCTION fProductInventoriesWithPreviousMonthCountsWithKPIs(@KPI INT)
    RETURNS TABLE
    AS
            RETURN
                SELECT TOP 100000
                    ProductName, InventoryDate, InventoryCount, PreviousMonthCount, CountVSPreviousCountKPI
                        FROM vProductInventoriesWithPreviousMonthCountsWithKPIs
                            WHERE CountVSPreviousCountKPI = @KPI
                                ORDER BY ProductName, MONTH(InventoryDate);
GO
```

Figure 1. Creation of a user-defined function to return a table based on a KPI set by the user.

If the user only wants to see a results set with a KPI of -1, for example, they can use the SQL code on the next page (Figure 2).

```
SELECT * FROM fProductInventoriesWithPreviousMonthCountsWithKPIs(-1);
```

Figure 2. The SELECT statement used to call the user-defined function using a KPI value of -1.

This will return only the rows with a KPI of -1 (Figure 3).

	ProductName	InventoryDate	InventoryCount	PreviousMonthCount	CountVSPreviousCountKPI
7	Chartreuse verte	March, 2017	59	79	-1
8	Chef Anton's Caj...	March, 2017	43	63	-1
9	Chocolade	March, 2017	5	25	-1
10	Côte de Blaye	March, 2017	7	27	-1
11	Escargots de Bo...	March, 2017	52	72	-1
12	Filo Mix	March, 2017	28	48	-1
13	Flotemysost	March, 2017	16	36	-1
14	Geitost	March, 2017	102	122	-1
15	Genen Shouyu	March, 2017	29	49	-1

Figure 3. The results table of a call on the function created to show rows from a view where KPI values are a parameter that can be set by the user.

Data engineers may also use user-defined functions for Extract-Transform-Load (ETL) processes. They can be used to change the formats of data, including splitting columns based on specified patterns, changing data types, and creating new columns.

User-defined functions can be useful when defining check constraints, as well. They can be used, for example, to prevent users from inputting invalid data without being themselves hard-coded. A check constraint with a UDF could prevent users from signing up for an event date in the past, for example, or for entering a date of birth that would make the individual in question old beyond possibility. Both of those check constraints would reference the current date and time.

Types of Function

Scalar functions

Scalar functions are used to return a single value. They can be used on the fly to calculate a single result, or can be used to populate a results table or a new table or view (Learn Microsoft 365, https://learn.microsoft.com/en-us/sql/relational-databases/user-defined-functions/user-defined-functions?view=sql-server-ver17, 2026)(External Site).

Scalar function frequently leverage parameters, which can be a variable that can be changed by users, or existing columns. The data engineer creating the scalar function will need to define

what data type is to be returned by the function. In the example below, a scalar function is created to calculate tips for dishes at a restaurant (Figure 4).

```
CREATE FUNCTION dbo.fRestaurantTips(@Value1 FLOAT, @Value2 FLOAT)
    RETURNS FLOAT
        AS
            BEGIN
                RETURN(SELECT @Value1 * (@Value2*0.01))
            END
GO
```

Figure 4. A simple scalar function used to return a single value, to be used to show tips on dishes at a restaurant.

The user can then call this function and set the parameters to the price of the dish, based on the column containing that price, and desired tip amounts (Figure 5).

```
SELECT
    OrderID
    ,DishName
    ,ProductCurrentPrice
    ,dbo.fRestaurantTips(ProductCurrentPrice, 10) AS TenPercentTip
    ,dbo.fRestaurantTips(ProductCurrentPrice, 15) AS FifteenPercentTip
    ,dbo.fRestaurantTips(ProductCurrentPrice, 20) AS TwentyPercentTip
    ,dbo.fRestaurantTips(ProductCurrentPrice, 25) AS TwentyFivePercentTip
    FROM RestaurantDishes;
```

Figure 5. The scalar function used in a SELECT statement to generate a results table showing different tip amounts on the dishes at a restaurant.

The results show the requested columns, including the new columns for each different tip value.

Results | Messages

	OrderID	DishName	ProductCurrentPrice	TenPercentTip	FifteenPercentTip	TwentyPercentTip	TwentyFivePercentTip
1	1	Pad Thai	15.00	1.5	2.25	3	3.75
2	2	Pizza	12.00	1.2	1.8	2.4	3
3	3	Breadstick	2.28	0.228	0.342	0.456	0.57
4	4	Lobster	39.00	3.9	5.85	7.8	9.75

Figure 6. The results of a SELECT statement to generate a results table showing dishes at a restaurant, their price, and different possible tip values.

A user could perform much more complicated calculations and use them to populate one or more columns in a table using scalar functions, without having the manually perform the calculation every time.

Inline Functions

Inline functions in SQL Server are used to return a table of results based on a parameter that can be changed by the user. The example user-defined function shown in Figure 1 is an example of a common inline table function. It can quickly be distinguished by the fact that it returns a table, and includes a parameter (@KPI) that can be set by the user when calling the function. An inline function using a parameter can often be replicated by a view using a WHERE clause. For example, the output of the fProductInventoriesWithPreviousMonthCountsWithKPIs function in Figure 1 for a particular KPI could be replicated by simply using a SELECT statement with a WHERE clause on the view vProductInventoriesWithPreviousMonthCountsWithKPIs with CountVSPreviousCountKPI equal to the desired KPI value.

Multi-Statement Functions

Multi-statement functions are used to return the results of multiple select statements encapsulated within a single function. This allows a user to return results into a single results table that originate in multiple tables based on a parameter they choose. (WiseOwlTutorials, Youtube, https://www.youtube.com/watch?v=nCAEgNxC7nU, 2026)(External Site)

This could be used, for example, to return plants from a Flora table and animals from a Fauna table based on the location at which they were found, where the species location is the parameter that can be changed in the multi-statement function (Figure 7).

```sql
CREATE FUNCTION dbo.fFloraAndFaunaAtLocation
    (@LocationSpecies nvarchar(100))
    RETURNS @LocationTable TABLE
        (SpeciesName nvarchar(100)
        ,SpeciesLocation nvarchar(100)
        ,SpeciesType nvarchar(100))
    AS
    BEGIN
        INSERT INTO @LocationTable
        SELECT
                PlantName
                ,PlantLocation
                ,'Plant'
                    FROM Flora
                        WHERE PlantLocation = @LocationSpecies
        INSERT INTO @LocationTable
        SELECT
                AnimalName
                ,AnimalLocation
                ,'Animal'
                    FROM Fauna
                        WHERE AnimalLocation = @LocationSpecies
        RETURN
    END
GO
```

Figure 7. Creation of a multi-statement function that allows users to create a results table with species of plants from a Plants table and animals from an Animals table based on their location.

The function can easily be called with a SELECT statement, with the parameter set to the desired species location (Figure 8).

```
SELECT * FROM dbo.fFloraAndFaunaAtLocation('Mount Rainier');
```

Figure 8. Calling the multi-statement function to return species found at Mount Rainier.

The results table shows both plants and animals, all at the location specified (Figure 9).

	SpeciesName	SpeciesLocation	SpeciesType
1	Beargrass	Mount Rainier	Plant
2	Rough paintbrush	Mount Rainier	Plant
3	Red columbine	Mount Rainier	Plant
4	Columbian monkshood	Mount Rainier	Plant
5	Matte saxifrage	Mount Rainier	Plant
6	Wolverine	Mount Rainier	Animal
7	Fisher	Mount Rainier	Animal
8	Moose	Mount Rainier	Animal
9	Blacktailed deer	Mount Rainier	Animal
10	Cascade fox	Mount Rainier	Animal

Figure 9. Results of the call on the function with the variable set to 'Mount Rainier'.

Multi-statement functions are useful in that they can greatly streamline workflows, allowing a user to execute a very simple function call instead of completing a complicated set of multiple SELECT statements every time they want to view information from multiple tables (SQLShack, https://www.sqlshack.com/sql-server-multi-statement-table-valued-functions/, 2026)(External Site).

Summary

Functions are a useful and versatile type of saved SQL code in SQL server. While SQL Server contains built-in functions, users can also create user-defined functions, designing functions that can perform many different calculations and accept parameters passed in by users. Data engineers can create scalar functions, which return a single value and are useful for populating columns; inline functions, which are used to populate results tables based on a parameter that can be changed by the user; and multi-statements functions, which can include SELECT statements that pull data from multiple tables based on a chosen parameter. SQL functions are an important tool a data engineer can use to save time and produce consistent, reliable results.